

ICT Trading Bot: Forex Edition

OANDA / MT5 CSV (also Binance USD-M) · branch `claude/keen-meitner-hlq76o` · 4 Oct 2026

1. Status

Written from scratch (the original `ict_fvg_bot.py` was not in the repo). Standard library only. 9 unit tests pass. It has **not** been run on real forex data, and the OANDA connection and order placement are untested.

2. Pipeline

Step	What it does
1. Daily bias	Three votes (+1 / -1): A previous-day high/low, B daily swing structure, C yesterday's liquidity sweep. Trade only when <code> score >= 2</code> . Locked once at the day's open. Buys only on bullish days, sells only on bearish days, nothing on neutral days.
2. 15m sweep	Wick beyond the lowest low (bullish) or highest high (bearish) of the previous 24 15m candles, with a close back inside.
3. 5m IFVG	Track FVGs of the opposite type. A candle <i>body</i> closing through the gap inverts it (wicks do not count). Inversion must come after the sweep candle closes.
4. Entry	Retest of the inverted zone with a rejection candle closing in the bias direction, inside a kill zone. Zone is dropped if price closes through it.
5. Risk	Stop 2 pips beyond the sweep extreme; trades with a stop under 5 pips are skipped. Target 1:3 R:R. One trade per day; open trades close at the day's end.

3. Forex-specific handling

Area	Behaviour
Trading day	17:00 to 17:00 New York time, DST-aware (22:00Z winter, 21:00Z summer). Bias locks at that open. Crypto keeps 00:00 UTC.
Weekends	Days with fewer than 100 5m candles are skipped, so weekend stubs are not counted as days.
Data	Only 5m candles are needed. 15m and daily candles are built from them on the same day boundaries, so a broker's daily alignment cannot skew the bias.
Kill zones	Entries only 02:00-05:00 or 07:00-10:00 New York time. <code>--killzones '2-5,7-10'</code> to change, <code>--killzones ''</code> to disable.
Pips	Inferred from symbol: JPY pairs 0.01, gold 0.1, otherwise 0.0001. <code>--pip</code> overrides.
Costs	<code>--spread-pips</code> is charged against each backtest trade in R.

4. Commands

Command	Purpose
<code>backtest --csv f.csv</code>	5m MT5/broker export in UTC; no network. Reports each bias method alone and the combined score: bias days, direction accuracy, trades, win rate, total R, average R. Same-candle stop and target counts as a stop.
<code>backtest --source oanda</code>	Pulls 5m candles from OANDA. Needs <code>OANDA_TOKEN</code> and <code>OANDA_ACCOUNT</code> .
<code>signal</code>	One-shot: today's votes, bias and setups.
<code>live</code>	Polls every 30s on an OANDA practice account (<code>--env live</code> for real). Logs signals; <code>--execute</code> sends one market order with stop and target attached. Size = <code>--risk-amount</code> / stop distance; use <code>--quote-rate</code> for non-account-currency quotes.

Example: `python ict_fvg_bot.py backtest --csv EURUSD_M5.csv --symbol EUR_USD --start 2025-01-01 --end 2026-01-01 --spread-pips 0.8`

5. Things to check

- **Vote A is nearly dead:** bias is taken at the day's open, and price there rarely sits outside yesterday's range except on gaps. With the combined score needing two agreeing votes, trades are rare. A real backtest will show whether it helps.
- **MT5 times:** MT5 usually exports in broker server time, not UTC. Convert before use or day boundaries and kill zones will be off.
- **Method C:** implemented as 'closed back beyond the swept level' rather than the spec's `close > d[-2].low`.
- **Body fully through:** body spans the whole gap. -- loose-body only needs a close beyond the far edge.
- **Before real funds:** backtest on real data, then OANDA practice dry-run, then practice with --execute.

Appendix: source (ict_fvg_bot.py)

```
#!/usr/bin/env python3
"""ICT bot for forex (OANDA / MT5 CSV) and Binance USD-M futures. Standard library only.

Pipeline: daily bias -> 15m liquidity sweep -> 5m IFVG retest -> entry (1:3 R:R), inside a kill zone.

Forex trading day = 17:00 New York to 17:00 New York (DST-aware); bias is locked at that open.
Crypto trading day = 00:00 UTC. Only 5m data is needed: 15m and daily candles are built from it
with the same day boundaries, so the data source's own daily alignment never matters.

# backtest on MT5/broker CSV export (no network needed)
python ict_fvg_bot.py backtest --market forex --symbol EUR_USD --csv EURUSD_M5.csv --start 2025-01-01 --end 2026-01-01 --
# backtest straight from OANDA (needs OANDA_TOKEN)
python ict_fvg_bot.py backtest --source oanda --symbol EUR_USD --start 2025-01-01 --end 2026-01-01
# today's bias / setups
python ict_fvg_bot.py signal --source oanda --symbol EUR_USD
# live loop on an OANDA practice account (dry-run unless --execute)
python ict_fvg_bot.py live --source oanda --symbol EUR_USD --risk-amount 50
"""
from __future__ import annotations

import argparse
import csv
import hashlib
import hmac
import json
import os
import time
import urllib.parse
import urllib.request
from bisect import bisect_left
from dataclasses import dataclass, field
from datetime import datetime, timedelta, timezone
from zoneinfo import ZoneInfo

MIN = 60_000
HOUR = 3_600_000
BULL, BEAR = 1, -1
UTC, NY = ZoneInfo("UTC"), ZoneInfo("America/New_York")

# ----- markets
@dataclass(frozen=True)
class Market:
    name: str
    tz: ZoneInfo
    anchor_hour: int # local hour at which the trading day starts
    killzones: tuple = () # ((start_hr, end_hr), ...) New York time; () = always allowed
    min_candles: int = 100 # 5m candles for a day to count (skips weekends/holidays)

    def day_start(self, t):
        local = datetime.fromtimestamp(t / 1000, self.tz)
        d = (local - timedelta(hours=self.anchor_hour)).date()
        return int(datetime(d.year, d.month, d.day, self.anchor_hour, tzinfo=self.tz).timestamp() * 1000)

    def day_end(self, start):
        d = datetime.fromtimestamp(start / 1000, self.tz).date() + timedelta(days=1)
        return int(datetime(d.year, d.month, d.day, self.anchor_hour, tzinfo=self.tz).timestamp() * 1000)

    def in_killzone(self, t):
        if not self.killzones:
            return True
        h = datetime.fromtimestamp(t / 1000, NY)
        x = h.hour + h.minute / 60
        return any(a <= x < b for a, b in self.killzones)

FOREX = Market("forex", NY, 17, killzones=((2, 5), (7, 10))) # London + New York AM
CRYPTO = Market("crypto", UTC, 0)
MARKETS = {"forex": FOREX, "crypto": CRYPTO}

def pip_size(symbol):
    s = symbol.upper()
    return 0.01 if "JPY" in s else 0.1 if s.startswith("XAU") else 0.0001

@dataclass
class Candle:
    t: int # open time, ms
    o: float
```

```

h: float
l: float
c: float

# ----- resampling
def resample(cs, key_fn, end_fn=None, until=None):
    """Group consecutive candles by key_fn(t). Drops a final bucket that ends after `until` (still forming)."""
    out, last = [], None
    for c in cs:
        k = key_fn(c.t)
        if k != last:
            out.append(Candle(k, c.o, c.h, c.l, c.c)); last = k
        else:
            b = out[-1]; b.h = max(b.h, c.h); b.l = min(b.l, c.l); b.c = c.c
    if until is not None and out and end_fn(out[-1].t) > until:
        out.pop()
    return out

def to_15m(c5, until=None):
    return resample(c5, lambda t: t - t % (15 * MIN), lambda k: k + 15 * MIN, until)

def to_daily(c5, market, until=None):
    """Daily candles on the market's day boundaries; thin days (weekend/holiday stubs) are dropped."""
    counts = {}
    for c in c5:
        k = market.day_start(c.t); counts[k] = counts.get(k, 0) + 1
    days = resample(c5, market.day_start, market.day_end, until)
    return [d for d in days if counts[d.t] >= market.min_candles]

# ----- sources
def _http(url, params=None, headers=None, method="GET", body=None):
    if params:
        url += "?" + urllib.parse.urlencode(params)
    req = urllib.request.Request(url, headers=headers or {}, method=method,
                                data=json.dumps(body).encode() if body is not None else None)
    with urllib.request.urlopen(req, timeout=20) as r:
        return json.load(r)

def load_csv(path):
    """5m candles from a CSV export (MT5 / broker / generic). Columns by header name, else positional.

    Accepted time forms: epoch seconds/ms, ISO-8601, 'YYYY.MM.DD HH:MM[:SS]' (MT5), or separate Date and Time
    columns. Times are UTC. Header is optional; positional order is time,open,high,low,close.
    """
    with open(path, newline="") as f:
        sample = f.read(4096); f.seek(0)
        dialect = csv.Sniffer().sniff(sample, delimiters=";\t")
        rows = [r for r in csv.reader(f, dialect) if r]
        head = [x.strip().lower().strip("<>") for x in rows[0]]
        named = "open" in head
        if named:
            col = {n: head.index(n) for n in ("open", "high", "low", "close")}
            tcols = [head.index(n) for n in ("date", "time") if n in head] or [head.index("timestamp") if "timestamp" in head else
            rows = rows[1:]
        else:
            col, tcols = {"open": 1, "high": 2, "low": 3, "close": 4}, [0]
        out = []
        for r in rows:
            ts = " ".join(r[i].strip() for i in tcols)
            out.append(Candle(_parse_time(ts), float(r[col["open"]]), float(r[col["high"]]), float(r[col["low"]]), float(r[col["c
        out.sort(key=lambda c: c.t)
        return out

def _parse_time(s):
    if s.replace(".", "", 1).isdigit() and "." not in s[:5]:
        v = float(s)
        return int(v if v > 1e11 else v * 1000)
    for fmt in ("%Y.%m.%d %H:%M:%S", "%Y.%m.%d %H:%M", "%Y-%m-%d %H:%M:%S", "%Y-%m-%d %H:%M", "%Y-%m-%dT%H:%M:%S", "%Y-%m-%dT
    try:
        return int(datetime.strptime(s[:19] if "T" in s else s, fmt).replace(tzinfo=UTC).timestamp() * 1000)
    except ValueError:
        pass
    raise ValueError(f"unrecognised time: {s!r}")

class Binance:

```

```

name = "binance"

def __init__(self, testnet=False):
    self.base = "https://testnet.binancefuture.com" if testnet else "https://fapi.binance.com"
    self.key, self.secret = os.environ.get("BINANCE_API_KEY", ""), os.environ.get("BINANCE_API_SECRET", "")

def fetch_5m(self, symbol, start, end):
    out, cur = [], start
    while cur < end:
        rows = _http(self.base + "/fapi/v1/klines", {"symbol": symbol, "interval": "5m", "startTime": cur, "endTime": end
        out += [Candle(r[0], float(r[1]), float(r[2]), float(r[3]), float(r[4])) for r in rows]
        if len(rows) < 1500:
            break
        cur = rows[-1][0] + 1
    return out

def place(self, symbol, su, a):
    q = urllib.parse.urlencode
    def signed(path, params):
        params = dict(params, timestamp=int(time.time() * 1000), recvWindow=5000)
        qs = q(params)
        sig = hmac.new(self.secret.encode(), qs.encode(), hashlib.sha256).hexdigest()
        return _http(f"{self.base}{path}?{qs}&signature={sig}", headers={"X-MBX-APIKEY": self.key}, method="POST")
    info = _http(self.base + "/fapi/v1/exchangeInfo")
    step = next(float(x["filterType"]: x for x in s["filters"])[ "LOT_SIZE" ][ "stepSize" ]) for s in info["symbols"] if s["
    qty = int(a.risk_amount / abs(su.entry - su.stop) / step) * step
    side, opp = ("BUY", "SELL") if su.direction == BULL else ("SELL", "BUY")
    common = dict(symbol=symbol, quantity=f"{qty:.8f}".rstrip("0"))
    signed("/fapi/v1/order", dict(common, side=side, type="MARKET"))
    signed("/fapi/v1/order", dict(common, side=opp, type="STOP_MARKET", stopPrice=f"{su.stop:.2f}", reduceOnly="true"))
    signed("/fapi/v1/order", dict(common, side=opp, type="TAKE_PROFIT_MARKET", stopPrice=f"{su.target:.2f}", reduceOnly="

class Oanda:
    """OANDA v20. Env: OANDA_TOKEN, OANDA_ACCOUNT. Practice account unless env='live'."""
    name = "oanda"

    def __init__(self, env="practice"):
        self.base = "https://api-fxpractice.oanda.com" if env == "practice" else "https://api-fxtrade.oanda.com"
        self.token, self.account = os.environ.get("OANDA_TOKEN", ""), os.environ.get("OANDA_ACCOUNT", "")
        self.h = {"Authorization": f"Bearer {self.token}", "Content-Type": "application/json"}

    def fetch_5m(self, symbol, start, end):
        out, cur = [], start
        while cur < end:
            d = _http(f"{self.base}/v3/instruments/{symbol}/candles",
            {"granularity": "M5", "price": "M", "from": f"{cur / 1000:.0f}", "count": 5000}, self.h)
            rows = [c for c in d["candles"] if c["complete"]]
            if not d["candles"]:
                break
            for c in rows:
                t = int(datetime.strptime(c["time"][:19], "%Y-%m-%dT%H:%M:%S").replace(tzinfo=UTC).timestamp() * 1000)
                if t < end:
                    m = c["mid"]; out.append(Candle(t, float(m["o"]), float(m["h"]), float(m["l"]), float(m["c"])))
            last = int(datetime.strptime(d["candles"][-1]["time"][:19], "%Y-%m-%dT%H:%M:%S").replace(tzinfo=UTC).timestamp())
            if len(d["candles"]) < 5000 or last + 5 * MIN <= cur:
                break
            cur = last + 5 * MIN
        return out

    def place(self, symbol, su, a):
        pip = a.pip or pip_size(symbol)
        dec = len(f"{pip:.10f}".rstrip("0").split(".")[1]) + 1
        units = int(a.risk_amount / (abs(su.entry - su.stop) * a.quote_rate)) * su.direction
        order = {"type": "MARKET", "instrument": symbol, "units": str(units), "timeInForce": "FOK", "positionFill": "DEFAULT"
        "stopLossOnFill": {"price": f"{su.stop:.{dec}f}"}, "takeProfitOnFill": {"price": f"{su.target:.{dec}f}"}}
        return _http(f"{self.base}/v3/accounts/{self.account}/orders", headers=self.h, method="POST", body={"order": order})

# ----- swings / FVG logic
def swing_highs(cs, n=2):
    return [i for i in range(n, len(cs) - n)
            if all(cs[i].h > cs[i - k].h and cs[i].h >= cs[i + k].h for k in range(1, n + 1))]

def swing_lows(cs, n=2):
    return [i for i in range(n, len(cs) - n)
            if all(cs[i].l < cs[i - k].l and cs[i].l <= cs[i + k].l for k in range(1, n + 1))]

@dataclass
class Zone:
    kind: int
    low: float

```

```

high: float
formed: int
inverted_at: int = -1

def fvg_at(cs, i):
    """FVG completed by candle i (pattern i-2, i-1, i), or None."""
    if i < 2:
        return None
    if cs[i].l > cs[i - 2].h:
        return Zone(BULL, cs[i - 2].h, cs[i].l, i)
    if cs[i].h < cs[i - 2].l:
        return Zone(BEAR, cs[i].h, cs[i - 2].l, i)
    return None

def inverts(zone, c, strict=True):
    """Candle body closes through the gap (wicks don't count). strict: body spans the whole gap."""
    if zone.kind == BEAR:
        return c.c > zone.high and (not strict or c.o <= zone.low)
    return c.c < zone.low and (not strict or c.o >= zone.high)

# ----- daily bias
def vote_pdh_pdl(d, price):
    return BULL if price > d[-1].h else BEAR if price < d[-1].l else 0

def vote_structure(d, n=2):
    hi, lo = swing_highs(d, n)[-2:], swing_lows(d, n)[-2:]
    if len(hi) < 2 or len(lo) < 2:
        return 0
    up = d[hi[1]].h > d[hi[0]].h and d[lo[1]].l > d[lo[0]].l
    dn = d[hi[1]].h < d[hi[0]].h and d[lo[1]].l < d[lo[0]].l
    return BULL if up else BEAR if dn else 0

def vote_sweep(d, lookback=5):
    if len(d) < lookback + 1:
        return 0
    prev, y = d[-lookback - 1:-1], d[-1]
    lo, hi = min(c.l for c in prev), max(c.h for c in prev)
    if y.l < lo and y.c > lo:
        return BULL
    if y.h > hi and y.c < hi:
        return BEAR
    return 0

METHODS = {"A_pdh_pdl": lambda d, p: vote_pdh_pdl(d, p),
           "B_structure": lambda d, p: vote_structure(d),
           "C_sweep": lambda d, p: vote_sweep(d)}
def daily_bias_votes(d, price):
    return {k: f(d, price) for k, f in METHODS.items()}

def daily_bias(d, price, min_score=2):
    """d = completed daily candles (d[-1] = previous day); price = price at the day's open."""
    score = sum(daily_bias_votes(d, price).values())
    return BULL if score >= min_score else BEAR if score <= -min_score else 0

# ----- 15m sweep -> 5m IFVG entry
@dataclass
class Setup:
    direction: int
    entry_t: int
    entry: float
    stop: float
    target: float
    sweep_extreme: float
    sweep_t: int
    zone: Zone = field(repr=False, default=None)

@dataclass
class Params:
    market: Market = FOREX
    sweep_lookback: int = 24
    fvg_wait: int = 48
    rr: float = 3.0
    pip: float = 0.0001

```

```

buffer_pips: float = 2.0          # forex: stop distance beyond the sweep extreme
buffer_frac: float = 0.0005       # crypto: same, as a fraction of price
min_risk_pips: float = 5.0        # forex: skip trades whose stop is tighter than this (spread eats them)
spread_pips: float = 0.0          # backtest cost per trade
strict_body: bool = True
max_trades_per_day: int = 1

def buffer(self, price):
    return self.buffer_pips * self.pip if self.market.name == "forex" else price * self.buffer_frac

def min_risk(self):
    return self.min_risk_pips * self.pip if self.market.name == "forex" else 0.0

def find_sweeps(c15, k0, k1, direction, p):
    for k in range(max(k0, p.sweep_lookback), k1):
        win, c = c15[k - p.sweep_lookback:k], c15[k]
        if direction == BULL:
            lvl = min(x.l for x in win)
            if c.l < lvl and c.c > lvl:
                yield k, lvl, c.l
        else:
            lvl = max(x.h for x in win)
            if c.h > lvl and c.c < lvl:
                yield k, lvl, c.h

def find_ifvg_entry(c5, i0, i1, direction, sweep_t, p):
    """FVG -> body-close inversion (after the sweep closes) -> retest rejection inside the kill zone."""
    sweep_close = sweep_t + 15 * MIN
    wanted = BEAR if direction == BULL else BULL
    active, inverted = [], []
    for j in range(i0, min(i1, i0 + p.fvg_wait)):
        c = c5[j]
        for z in inverted[:]:
            if direction == BULL:
                if c.c < z.low:
                    inverted.remove(z); continue
                if c.l <= z.high and c.c > z.high and c.c > c.o and p.market.in_killzone(c.t):
                    return c, z
            else:
                if c.c > z.high:
                    inverted.remove(z); continue
                if c.h >= z.low and c.c < z.low and c.c < c.o and p.market.in_killzone(c.t):
                    return c, z
        if c.t >= sweep_close:
            for z in active[:]:
                if inverts(z, c, p.strict_body):
                    active.remove(z); z.inverted_at = j; inverted.append(z)
        z = fvg_at(c5, j)
        if z and z.kind == wanted and c5[j - 2].t >= sweep_t:
            active.append(z)
    return None

def find_setups(c15, c5, day_start, day_end, direction, p):
    """Setups for one trading day, earliest first. direction = locked daily bias."""
    if direction == 0:
        return []
    t15, t5 = [c.t for c in c15], [c.t for c in c5]
    k0, k1 = bisect_left(t15, day_start), bisect_left(t15, day_end)
    i_end = bisect_left(t5, day_end)
    out, last_entry_t = [], -1
    for k, lvl, extreme in find_sweeps(c15, k0, k1, direction, p):
        sweep_t = c15[k].t
        hit = find_ifvg_entry(c5, bisect_left(t5, sweep_t), i_end, direction, sweep_t, p)
        if not hit:
            continue
        c, z = hit
        if c.t <= last_entry_t:
            continue
        entry, buf = c.c, p.buffer(c.c)
        stop = extreme - buf if direction == BULL else extreme + buf
        risk = (entry - stop) * direction
        if risk <= 0 or risk < p.min_risk():
            continue
        out.append(Setup(direction, c.t, entry, stop, entry + direction * p.rr * risk, extreme, sweep_t, z))
        last_entry_t = c.t
    out.sort(key=lambda s: s.entry_t)
    return out

def simulate(setup, c5, day_end):
    """(R multiple before costs, reason). Same-candle SL+TP -> SL. Open trades close at the day's end."""

```

```

d, risk, last = setup.direction, (setup.entry - setup.stop) * setup.direction, None
t5 = [c.t for c in c5]
for c in c5[bisect_left(t5, setup.entry_t + 5 * MIN):]:
    if c.t >= day_end:
        break
    last = c
    if (c.l <= setup.stop) if d == BULL else (c.h >= setup.stop):
        return -1.0, "SL"
    if (c.h >= setup.target) if d == BULL else (c.l <= setup.target):
        return (setup.target - setup.entry) * d / risk, "TP"
if last is None:
    return 0.0, "NOFILL"
return (last.c - setup.entry) * d / risk, "EOD"

# ----- backtest
def backtest(daily, c15, c5, p, start_ms=0, end_ms=None, warmup=12, fee_r=0.0):
    """Same pipeline gated by each bias method alone and by the combined score."""
    variants = list(METHODS) + ["combined"]
    stats = {v: dict(bias_days=0, bias_right=0, trades=0, wins=0, r=0.0) for v in variants}
    log = []
    for i in range(warmup, len(daily)):
        today = daily[i]
        if today.t < start_ms or (end_ms and today.t >= end_ms):
            continue
        end = p.market.day_end(today.t)
        hist, price = daily[:i], today.o # bias locked at the day's open
        votes = daily_bias_votes(hist, price)
        biases = dict(votes, combined=daily_bias(hist, price))
        actual = BULL if today.c > today.o else BEAR
        for v in variants:
            b, s = biases[v], stats[v]
            if b == 0:
                continue
            s["bias_days"] += 1; s["bias_right"] += (b == actual)
            for su in find_setups(c15, c5, today.t, end, b, p)[:p.max_trades_per_day]:
                r, why = simulate(su, c5, end)
                r -= fee_r + p.spread_pips * p.pip / ((su.entry - su.stop) * su.direction)
                s["trades"] += 1; s["wins"] += r > 0; s["r"] += r
            if v == "combined":
                log.append((datetime.fromtimestamp(su.entry_t / 1000, UTC), b, su.entry, su.stop, su.target, why, r))
    return stats, log

def print_report(stats, log):
    print(f"{'bias method':<14}{'bias days':>10}{'dir acc':>9}{'trades':>8}{'win%':>7}{'total R':>9}{'avg R':>8}")
    for v, s in stats.items():
        acc = s["bias_right"] / s["bias_days"] * 100 if s["bias_days"] else 0
        wr = s["wins"] / s["trades"] * 100 if s["trades"] else 0
        avg = s["r"] / s["trades"] if s["trades"] else 0
        print(f"v:{<14}{s['bias_days']:>10}{acc:>8.1f}%{s['trades']:>8}{wr:>6.1f}%{s['r']:>9.2f}{avg:>8.2f}")
    print("\ncombined-score trades (UTC):")
    for t, b, e, sl, tp, why, r in log:
        print(f" {t:%Y-%m-%d %H:%M} {'LONG ' if b == BULL else 'SHORT'} entry={e:.5f} sl={sl:.5f} tp={tp:.5f} {why:<5} {r:+.5f}")

# ----- data + live
def get_broker(a):
    return Oanda(getattr(a, "env", "practice")) if a.source == "oanda" else Binance(getattr(a, "testnet", False))

def make_params(a):
    m = MARKETS[a.market]
    if a.killzones is not None:
        kz = tuple(tuple(float(x) for x in z.split("-")) for z in a.killzones.split(",")) if a.killzones else ()
        m = Market(m.name, m.tz, m.anchor_hour, kz, m.min_candles)
    p = Params(market=m, rr=a.rr, pip=a.pip or pip_size(a.symbol), spread_pips=a.spread_pips,
               strict_body=not a.loose_body, min_risk_pips=a.min_risk_pips)
    return p

def day_state(broker, symbol, p, now_ms):
    """Completed-day history + today's 15m/5m candles. Returns None until today's first candle exists."""
    ds = p.market.day_start(now_ms)
    hist = broker.fetch_5m(symbol, ds - 40 * 24 * HOUR, ds)
    today = broker.fetch_5m(symbol, ds - 12 * HOUR, now_ms)
    until = (today[-1].t + 5 * MIN) if today else None
    c5 = [c for c in today if c.t + 5 * MIN <= now_ms]
    first = next((c for c in c5 if c.t >= ds), None)
    if not first:
        return None
    return to_daily(hist, p.market), to_15m(c5, until), c5, ds, first.o

```

```

def run_live(a):
    p, broker = make_params(a), get_broker(a)
    locked, traded = {}, set()
    while True:
        now = int(time.time()) * 1000
        ds = p.market.day_start(now)
        st = day_state(broker, a.symbol, p, now)
        if st:
            daily, c15, c5, ds, day_open = st
            if ds not in locked:
                locked[ds] = daily_bias(daily, day_open)
                print(f"[{datetime.fromtimestamp(ds / 1000, UTC):%Y-%m-%d %H:%M}Z] bias locked "
                    f"{ 'BULL: 'BULLISH', BEAR: 'BEARISH', 0: 'NEUTRAL'}[locked[ds]] } {daily_bias_votes(daily, day_open)}")
            for su in find_setups(c15, c5, ds, p.market.day_end(ds), locked[ds], p)[: p.max_trades_per_day]:
                if su.entry_t in traded or su.entry_t < now - 10 * MIN:
                    continue
                traded.add(su.entry_t)
                print("SIGNAL", su)
                if a.execute:
                    print(broker.place(a.symbol, su, a))
            time.sleep(30)

def ts(s):
    return int(datetime.strptime(s, "%Y-%m-%d").replace(tzinfo=UTC).timestamp() * 1000)

def main():
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    sub = ap.add_subparsers(dest="cmd", required=True)
    def common(sp):
        sp.add_argument("--market", choices=list(MARKETS), default=None, help="default: forex for oanda/csv, crypto for binan")
        sp.add_argument("--source", choices=["oanda", "binance"], default="oanda")
        sp.add_argument("--symbol", default="EUR_USD", help="OANDA EUR_USD style; Binance BTCUSDT")
        sp.add_argument("--pip", type=float, default=0.0, help="override pip size")
        sp.add_argument("--rr", type=float, default=3.0)
        sp.add_argument("--spread-pips", type=float, default=0.0)
        sp.add_argument("--min-risk-pips", type=float, default=5.0)
        sp.add_argument("--killzones", default=None, help="NY-time windows e.g. '2-5,7-10'; '' disables")
        sp.add_argument("--loose-body", action="store_true", help="IFVG needs only a close beyond the gap")
        sp.add_argument("--env", choices=["practice", "live"], default="practice", help="OANDA environment")
        sp.add_argument("--testnet", action="store_true", help="Binance testnet")
    b = sub.add_parser("backtest"); common(b)
    b.add_argument("--csv", help="5m CSV (MT5/broker export, UTC); skips the network")
    b.add_argument("--start", required=True); b.add_argument("--end", required=True)
    b.add_argument("--fee-r", type=float, default=0.0)
    s = sub.add_parser("signal"); common(s)
    l = sub.add_parser("live"); common(l)
    l.add_argument("--execute", action="store_true", help="place orders (default: log signals only)")
    l.add_argument("--risk-amount", type=float, default=10.0, help="account-currency risk per trade")
    l.add_argument("--quote-rate", type=float, default=1.0, help="account currency per 1 unit of quote currency")
    a = ap.parse_args()
    a.market = a.market or ("crypto" if a.source == "binance" else "forex")
    p = make_params(a)

    if a.cmd == "backtest":
        s0, e0 = ts(a.start), ts(a.end)
        if a.csv:
            c5 = load_csv(a.csv)
        else:
            c5 = get_broker(a).fetch_5m(a.symbol, s0 - 45 * 24 * HOUR, e0)
            c5 = [c for c in c5 if c.t < e0 + 2 * 24 * HOUR]
            stats, log = backtest(to_daily(c5, p.market), to_15m(c5), c5, p, start_ms=s0, end_ms=e0, fee_r=a.fee_r)
            print(f"{a.symbol} {a.start}..{a.end} market={a.market} pip={p.pip} spread={p.spread_pips}p "
                f"killzones={p.market.killzones or 'off'} strict_body={p.strict_body} candles5m={len(c5)}")
            print_report(stats, log)
        elif a.cmd == "signal":
            now = int(time.time()) * 1000
            st = day_state(get_broker(a), a.symbol, p, now)
            if not st:
                return print("no candle yet for today's session")
            daily, c15, c5, ds, day_open = st
            bias = daily_bias(daily, day_open)
            print("votes:", daily_bias_votes(daily, day_open), "->", {BULL: "BULLISH", BEAR: "BEARISH", 0: "NEUTRAL"}[bias])
            for su in find_setups(c15, c5, ds, p.market.day_end(ds), bias, p):
                print(su)
        else:
            run_live(a)

if __name__ == "__main__":
    main()

```